

Información Técnica y Estructura del Proyecto "Seguros"

Documento: Prueba Técnica Práctica

La aplicación está configurada para ser utilizada en modo web y con una API REST.

- **Nombre del Proyecto:** Seguros
- **Web de Tratamiento de *Endpoint*:** <https://www.jdoodle.com/spring>
- **API Rest (URL):** <https://v7izcwvncw.advcompiler.jdoodle.com/>
- **API Key (Clave Importante):**
1234567890
- **Repositorio Web (GitHub):** <https://github.com/oscardo/Seguros>

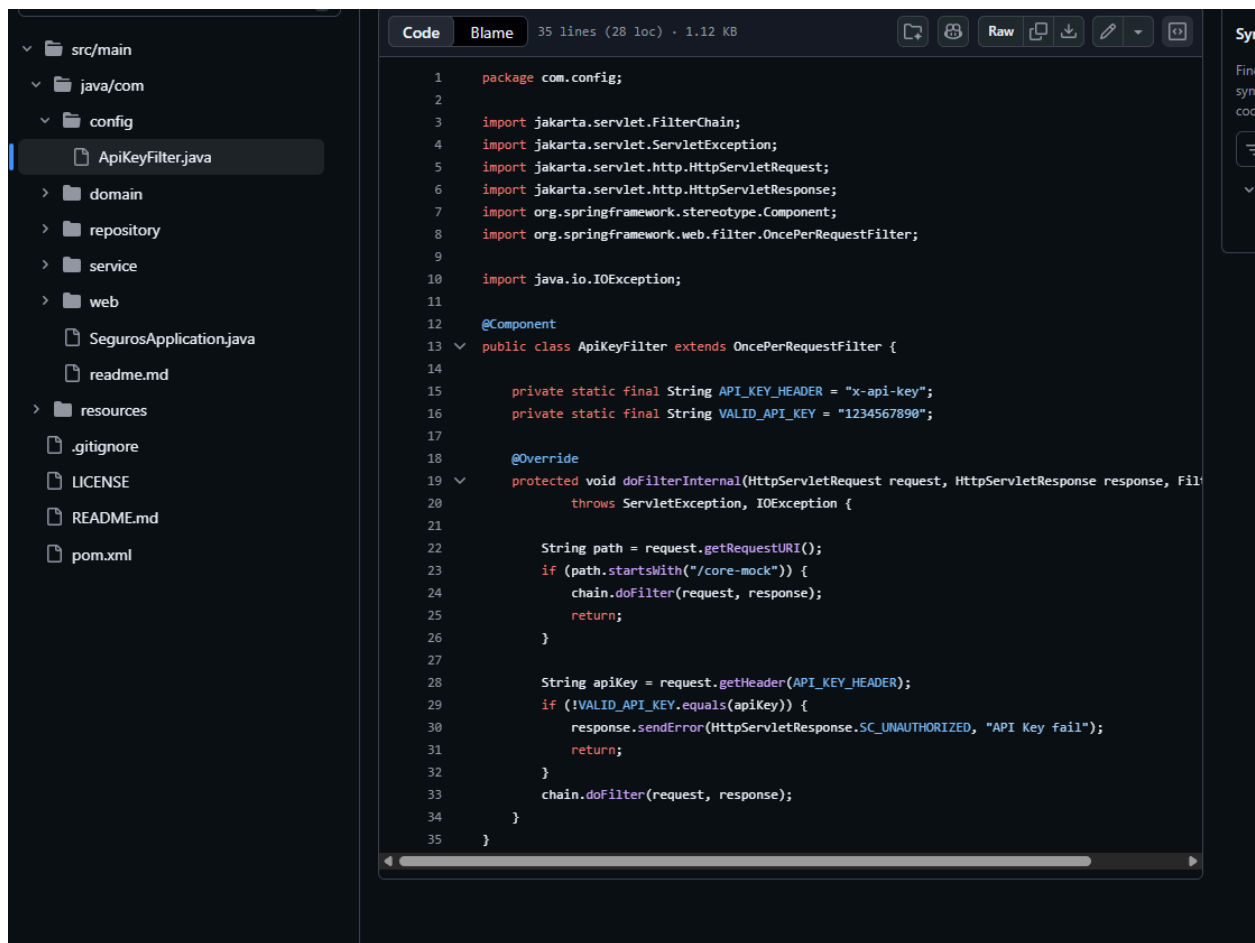
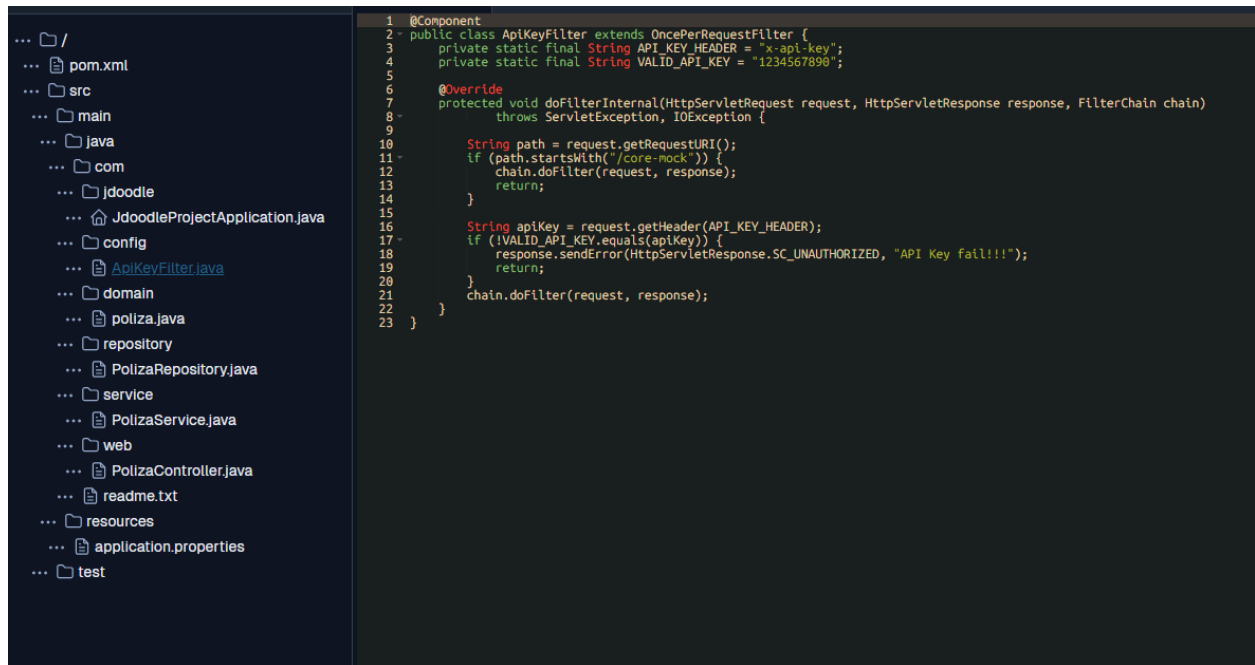
3. Estructura del Proyecto (src/main/java/com/seguros/)

Módulo/Carpeta	Propósito/Descripción
config/	Configuración (Incluye la Seguridad, específicamente el API Key Filter)
domain/	Domain (Contiene las Entidades ricas y Enums , que son el Core de negocio)
repository/	Repository (Implementa Spring Data JPA)
service/	Service (Representa los Casos de uso , incluyendo la Orquestación y el cliente Mock)
web/	Web (Contiene los Controllers y DTOs (Records))

src/main/java/com/seguros/

```
|— config/
|— domain/
|— repository/
|— service/
|— web/
```

Configuración (Seguridad (API Key Filter))



src/main

java/com

config

ApiKeyFilter.java

domain

EstadoPoliza.java

EstadoRiesgo.java

PasoProceso.java

Poliza.java

Riesgo.java

TipoPoliza.java

repository

service

web

SegurosApplication.java

readme.md

resources

.gitignore

LICENSE

README.md

pom.xml

CodeBlame

7 lines (6 loc) · 90 Bytes

1package com.domain;

2

3public enum EstadoPoliza {

4 ACTIVA,

5 RENOVADA,

6 CANCELADA

7}

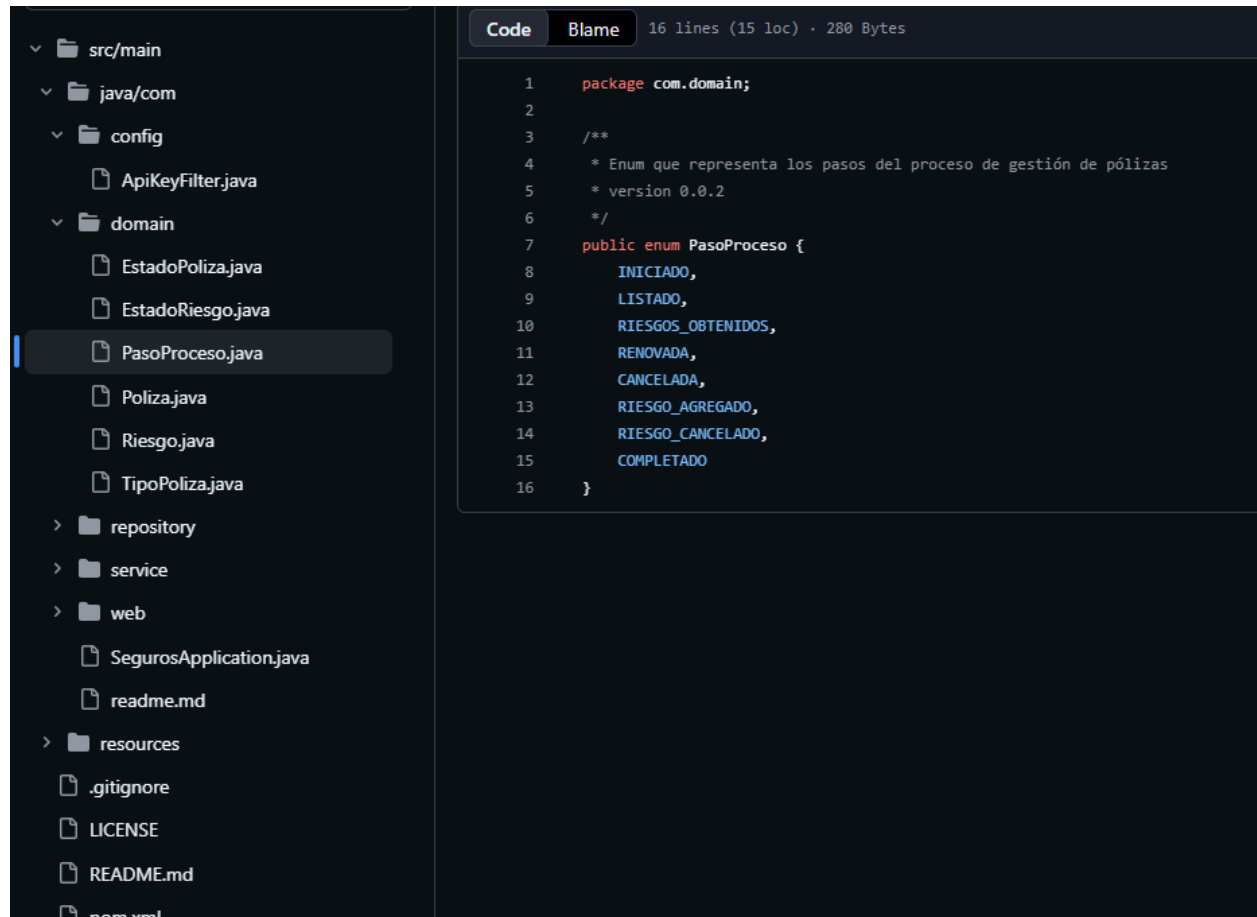
- src/main
 - java/com
 - config
 - ApiKeyFilter.java
 - domain
 - EstadoPoliza.java
 - EstadoRiesgo.java
 - PasoProceso.java
 - Poliza.java
 - Riesgo.java
 - TipoPoliza.java
 - repository
 - service
 - web
 - SegurosApplication.java
 - readme.md
 - resources
 - .gitignore
 - LICENSE
 - README.md
 - pom.xml

Code

Blame

6 lines (5 loc) · 76 Bytes

```
1 package com.domain;
2
3 public enum EstadoRiesgo {
4     ACTIVO,
5     CANCELADO
6 }
```



- src/main
 - java/com
 - config
 - ApiKeyFilter.java
 - domain
 - EstadoPoliza.java
 - EstadoRiesgo.java
 - PasoProceso.java
 - Poliza.java
 - Riesgo.java
 - TipoPoliza.java
 - repository
 - service
 - web
 - SegurosApplication.java
 - readme.md
 - resources
 - .gitignore
 - LICENSE
 - README.md
 - pom.xml

Code Blame 154 lines (130 loc) · 4.64 KB

```

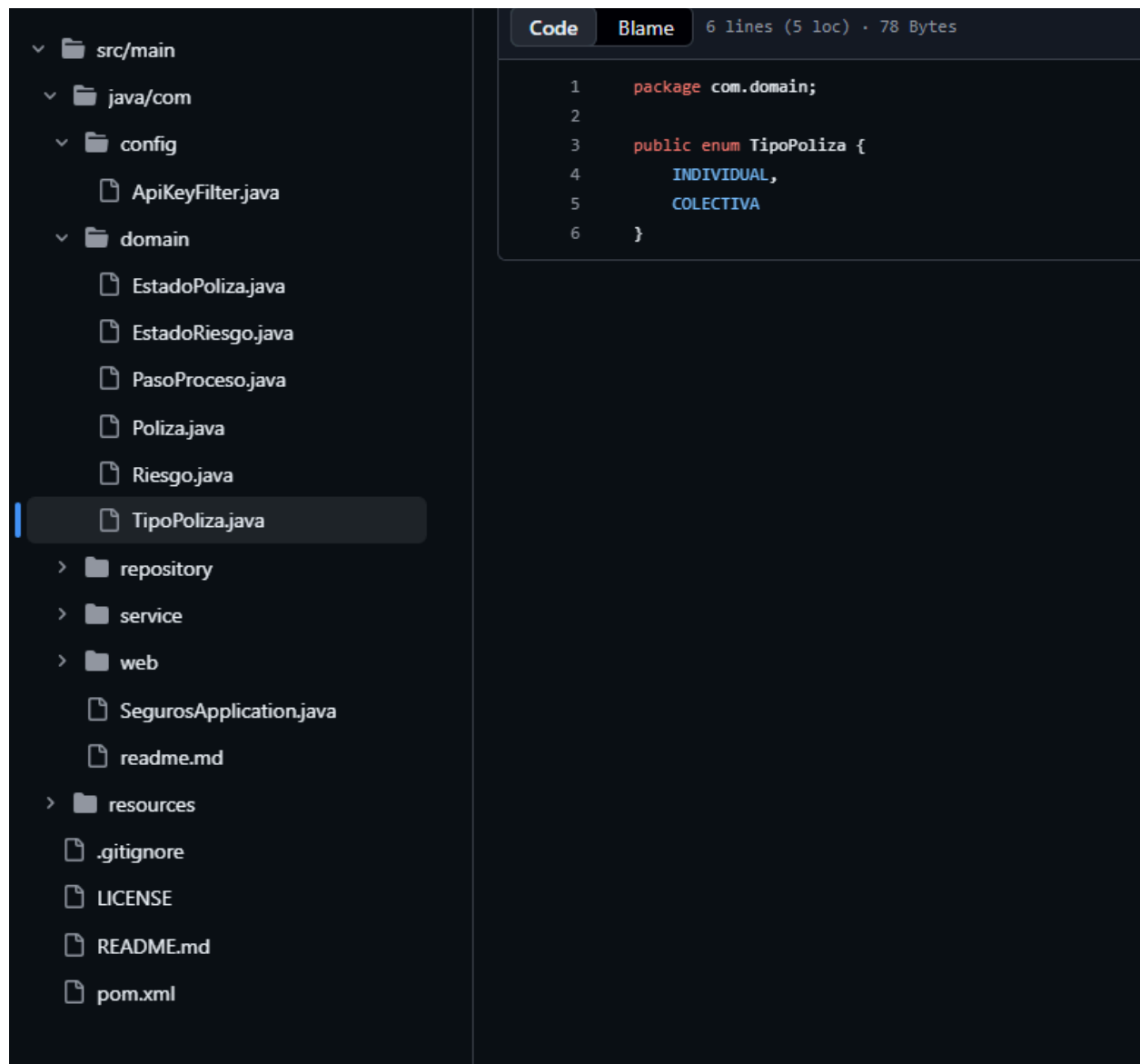
1  package com.domain;
2
3  import jakarta.persistence.*;
4  import lombok.Getter;
5  import lombok.NoArgsConstructor;
6  import lombok.Setter;
7
8  import java.math.BigDecimal;
9  import java.time.LocalDate;
10 import java.time.LocalTime;
11 import java.util.ArrayList;
12 import java.util.List;
13
14 @Entity
15 @Getter
16 @Setter
17 @NoArgsConstructor
18 public class Poliza {
19
20     @Id
21     @GeneratedValue(strategy = GenerationType.IDENTITY)
22     private Long id;
23
24     @Enumerated(EnumType.STRING)
25     private TipoPoliza tipo;
26
27     @Enumerated(EnumType.STRING)
28     private EstadoPoliza estado;
29
30     private BigDecimal canon;
31
32     private BigDecimal prima;
33
34     // Campos de fecha - version 0.0.2
35     private LocalDate fechaInicio;
36
37     private LocalDate fechaFin;
38
39     private Integer diaEnvioInformacion;
40
41     // Campo de hora de tráfico - version 0.0.2
42     private LocalTime horaTráfico;
43
44     // Seguimiento del paso del proceso - version 0.0.2
45     @Enumerated(EnumType.STRING)
46     private PasoProceso pasoProceso = PasoProceso.INICIADO;
47
48     @OneToMany(mappedBy = "poliza", cascade = CascadeType.ALL, orphanRemoval = true)
49     private List<Riesgo> riesgos = new ArrayList<>();
50
51     public Poliza(TipoPoliza tipo, BigDecimal canon, BigDecimal prima, EstadoPoliza estado,
52                 LocalDate fechaInicio, LocalDate fechaFin, Integer diaEnvioInformacion) {
53         this.tipo = tipo;
54         this.canon = canon;
55         this.prima = prima;
56         this.estado = estado;
57         this.fechaInicio = fechaInicio;
58         this.fechaFin = fechaFin;
59         this.diaEnvioInformacion = diaEnvioInformacion;
60     }
61
62     /**
63      * Paso 1: Listar pólizas por tipo y estado
64      */
65     public void listar(TipoPoliza tipo, EstadoPoliza estado) {
66         this.pasoProceso = PasoProceso.LISTADO;
67     }
68
69     /**
70      * Paso 2: Obtener riesgos de la póliza
71      */
72     public void obtenerRiesgos() {
73         if (this.pasoProceso.ordinal() < PasoProceso.LISTADO.ordinal()) {
74             throw new IllegalStateException("Debe listar la póliza primero");
75         }
76         this.pasoProceso = PasoProceso.RIESGOS_OBTENIDOS;
77     }
78
79     /**
80      * Paso 3: Renovar póliza (+IPC), validando estados
81      */
82     public void renovar(BigDecimal ipc) {
83         if (this.pasoProceso.ordinal() < PasoProceso.RIESGOS_OBTENIDOS.ordinal()) {
84             throw new IllegalStateException("Debe obtener riesgos antes de renovar");
85         }
86         if (this.estado == EstadoPoliza.CANCELADA) {

```

- src/main
 - java/com
 - config
 - ApiKeyFilter.java
 - domain
 - EstadoPoliza.java
 - EstadoRiesgo.java
 - PasoProceso.java
 - Poliza.java
 - Riesgo.java
 - TipoPoliza.java
 - repository
 - service
 - web
 - SegurosApplication.java
 - readme.md
 - resources
 - .gitignore
 - LICENSE
 - README.md
 - pom.xml

Code Blame 37 lines (28 loc) · 756 Bytes

```
1 package com.domain;
2
3 import jakarta.persistence.*;
4 import lombok.Getter;
5 import lombok.NoArgsConstructor;
6 import lombok.Setter;
7
8 import java.time.LocalDateTime;
9
10 @Entity
11 @Getter
12 @Setter
13 @NoArgsConstructor
14 public class Riesgo {
15
16     @Id
17     @GeneratedValue(strategy = GenerationType.IDENTITY)
18     private Long id;
19
20     @Enumerated(EnumType.STRING)
21     private EstadoRiesgo estado = EstadoRiesgo.ACTIVO;
22
23     @ManyToOne(fetch = FetchType.LAZY)
24     @JoinColumn(name = "poliza_id")
25     private Poliza poliza;
26
27     // Campo de hora de tráfico - version 0.0.2
28     private LocalDateTime horaTráfico;
29
30     public void vincularPoliza(Poliza poliza) {
31         this.poliza = poliza;
32     }
33
34     public void cancelar() {
35         this.estado = EstadoRiesgo.CANCELADO;
36     }
37 }
```



Domain (Entidades ricas y Enums (Core de negocio))

Unable to Sync the file with server!

```
... /
... pom.xml
... src
... main
... java
... com
... jdoodle
... JdoodleProjectApplication.java
... config
... ApiKeyFilter.java
... domain
... poliza.java
... repository
... PolizaRepository.java
... service
... PolizaService.java
... web
... PolizaController.java
... readme.txt
... resources
... application.properties
... test
```

```
1 @Entity @Getter @NoArgsConstructor
2 public class Poliza {
3     // TipoPoliza (INDIVIDUAL, COLECTIVA)
4     // EstadoPoliza (ACTIVA, RENOVADA, CANCELADA)
5     // EstadoRiesgo (ACTIVO, CANCELADO)
6     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id; //ID lógica
8     @Enumerated(EnumType.STRING) private TipoPoliza tipo; //public to private
9     @Enumerated(EnumType.STRING) private EstadoPoliza estado; //public to private
10    private BigDecimal canon; //$$$
11    private BigDecimal prima; //S
12
13    @OneToMany(mappedBy = "poliza", cascade = CascadeType.ALL)
14    private List<Riesgo> riesgos = new ArrayList<>(); //list new stack fifo
15
16    // REGLA: Renovar (+IPC), validando estados
17    public void renovar(BigDecimal ipc) {
18        if (this.estado == EstadoPoliza.CANCELADA) //no se debe
19            throw new IllegalStateException("No se puede renovar una póliza cancelada");
20
21        BigDecimal factor = BigDecimal.ONE.add(ipc); //aplicar
22        this.canon = this.canon.multiply(factor);
23        this.prima = this.prima.multiply(factor);
24        this.estado = EstadoPoliza.RENOVADA; //tres estados
25    }
26
27    public void cancelar() {
28        this.estado = EstadoPoliza.CANCELADA;
29        this.riesgos.forEach(Riesgo::cancelar); // REGLA: Cancelar poliza cancela sus riesgos
30    }
31
32    public void agregarRiesgo(Riesgo riesgo) {
33        // individual solo 1 riesgo, agregar exige validacion
34        if (this.tipo == TipoPoliza.INDIVIDUAL && !this.riesgos.isEmpty())
35            throw new IllegalStateException("Una póliza individual solo admite 1 riesgo");
36
37        //else{
38        //    //
39        //}
40        riesgo.vincularPoliza(this); //
41        this.riesgos.add(riesgo);
42    }
43 }
44
45 @Entity @Getter @NoArgsConstructor
46 public class Riesgo {
47     @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
48     private Long id;
49     @Enumerated(EnumType.STRING) private EstadoRiesgo estado = EstadoRiesgo.ACTIVO;
50
51     @ManyToOne(fetch = FetchType.LAZY)
52     @JoinColumn(name = "poliza_id")
53     private Poliza poliza;
54
55     public void vincularPoliza(Poliza poliza) { this.poliza = poliza; }
56     public void cancelar() { this.estado = EstadoRiesgo.CANCELADO; }
57 }
58
59
60
61
```

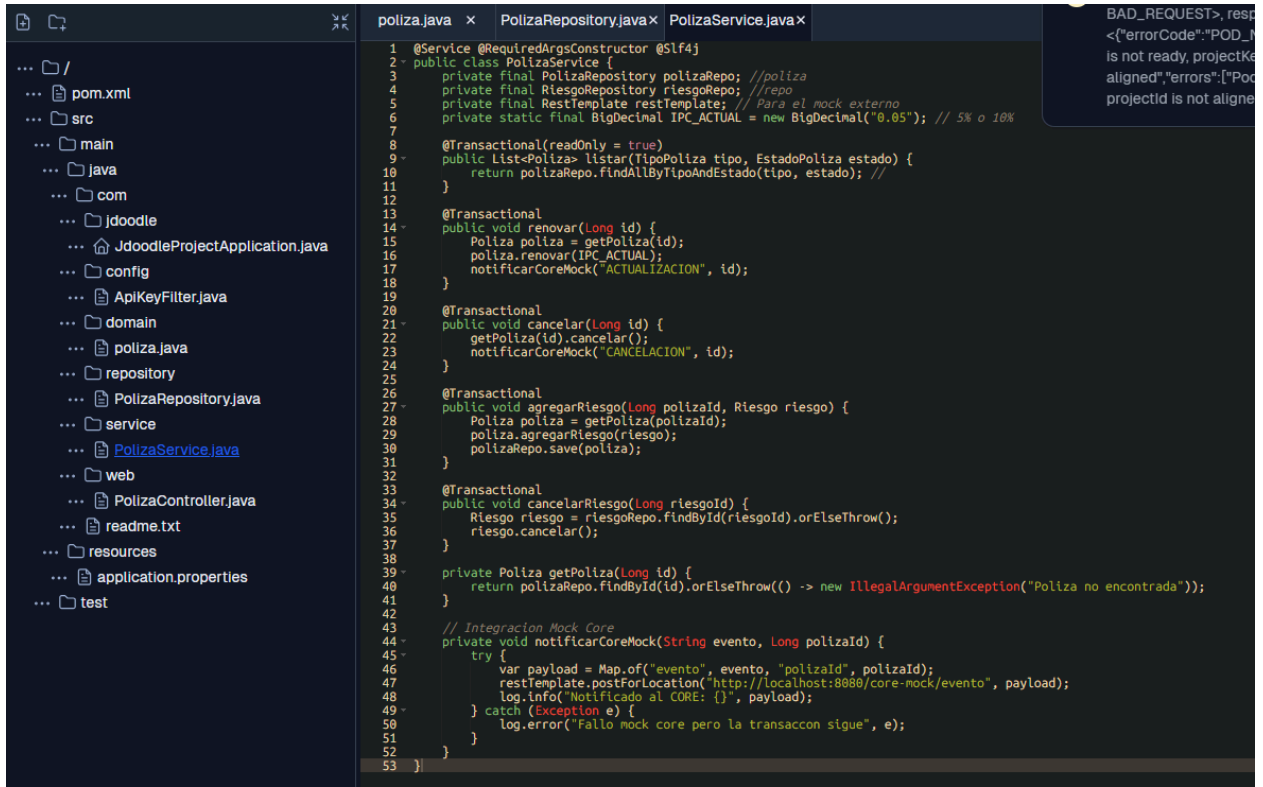
is not ready
aligned", "er
projectId is

Repository (Spring Data JPA)

```
... /
... pom.xml
... src
... main
... java
... com
... jdoodle
... JdoodleProjectApplication.java
... config
... ApiKeyFilter.java
... domain
... poliza.java
... repository
... PolizaRepository.java
... service
... PolizaService.java
... web
... PolizaController.java
... readme.txt
... resources
... application.properties
... test
```

```
1 //esta informacion es relevante por progrmaaria
2 public interface PolizaRepository extends JpaRepository<Poliza, Long> {
3     List<Poliza> findAllByTipoAndEstado(TipoPoliza tipo, EstadoPoliza estado);
4 }
5 public interface RiesgoRepository extends JpaRepository<Riesgo, Long> {}
```

Service (Casos de uso (Orquestación y cliente Mock))



```
1 @Service @RequiredArgsConstructor @Slf4j
2 public class PolizaService {
3     private final PolizaRepository polizaRepo; //poliza
4     private final RiesgoRepository riesgoRepo; //riesgo
5     private final RestTemplate restTemplate; // Para el mock externo
6     private static final BigDecimal IPC_ACTUAL = new BigDecimal("0.05"); // 5% o 10%
7
8     @Transactional(readOnly = true)
9     public List<Poliza> listar(TipoPoliza tipo, EstadoPoliza estado) {
10         return polizaRepo.findAllByTipoAndEstado(tipo, estado); //
11     }
12
13     @Transactional
14     public void renovar(Long id) {
15         Poliza poliza = getPoliza(id);
16         poliza.renovar(IPC_ACTUAL);
17         notificarCoreMock("ACTUALIZACION", id);
18     }
19
20     @Transactional
21     public void cancelar(Long id) {
22         getPoliza(id).cancelar();
23         notificarCoreMock("CANCELACION", id);
24     }
25
26     @Transactional
27     public void agregarRiesgo(Long polizaId, Riesgo riesgo) {
28         Poliza poliza = getPoliza(polizaId);
29         poliza.agregarRiesgo(riesgo);
30         polizaRepo.save(poliza);
31     }
32
33     @Transactional
34     public void cancelarRiesgo(Long riesgoId) {
35         Riesgo riesgo = riesgoRepo.findById(riesgoId).orElseThrow();
36         riesgo.cancelar();
37     }
38
39     private Poliza getPoliza(Long id) {
40         return polizaRepo.findById(id).orElseThrow(() -> new IllegalArgumentException("Poliza no encontrada"));
41     }
42
43     // Integracion Mock Core
44     private void notificarCoreMock(String evento, Long polizaId) {
45         try {
46             var payload = Map.of("evento", evento, "polizaId", polizaId);
47             restTemplate.postForLocation("http://localhost:8080/core-mock/evento", payload);
48             log.info("Notificado al CORE: {}", payload);
49         } catch (Exception e) {
50             log.error("Fallo mock core pero la transaccion sigue", e);
51         }
52     }
53 }
```

Web (Controllers y DTOs (Records))

main

+

Q

Go to file

t

src/main

java/com

config

domain

repository

service

web

ApiKeyFilter.java

EstadoPoliza.java

EstadoRiesgo.java

PasoProceso.java

Poliza.java

Riesgo.java

TipoPoliza.java

CoreMockController.java

PolizaController.java

RiesgoController.java

SegurosApplication.java

readme.md

resources

.gitignore

LICENSE

README.md

pom.xml

oscardo version 0.0.1

Code

Blame

17 lines (13 loc) · 449 Bytes

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

```
package com.web;

import lombok.extern.slf4j.Slf4j;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/core-mock")
@Slf4j
public class CoreMockController {

    public record CoreEvent(String evento, Long polizaId) {}

    @PostMapping("/evento")
    public void recibirEvento(@RequestBody CoreEvent event) {
        log.info("CORE MOCK RECIBIÓ: Evento={}, PolizaId={}", event.evento(), event.polizaId());
    }
}
```



```

1  package com.web;
2
3  import com.domain.Poliza;
4  import com.domain.Riesgo;
5  import com.domain.TipoPoliza;
6  import com.service.PolizaService;
7  import lombok.RequiredArgsConstructor;
8  import org.springframework.http.ResponseEntity;
9  import org.springframework.web.bind.annotation.*;
10
11  import java.util.List;
12
13  @RestController
14  @RequestMapping("/polizas")
15  @RequiredArgsConstructor
16  public class PolizaController {
17
18      private final PolizaService service;
19
20      /**
21       * Paso 1: Listar pólizas por tipo y estado
22       */
23      @GetMapping
24      public List<Poliza> listar(
25          @RequestParam TipoPoliza tipo,
26          @RequestParam String estado) {
27          return service.listar(tipo, estado);
28      }
29
30      /**
31       * Paso 2: Obtener riesgos de una póliza
32       */
33      @GetMapping("/{id}/riesgos")
34      public List<Riesgo> obtenerRiesgos(@PathVariable Long id) {
35          return service.obtenerRiesgosDePoliza(id);
36      }
37
38      /**
39       * Paso 3: Renovar póliza (+IPC 5%)
40       */
41      @PostMapping("/{id}/renovar")
42      public ResponseEntity<Poliza> renovar(@PathVariable Long id) {
43          Poliza poliza = service.renovar(id);
44          return ResponseEntity.ok(poliza);
45      }
46
47      /**
48       * Paso 4: Cancelar póliza (y sus riesgos)
49       */
50      @PostMapping("/{id}/cancelar")
51      public ResponseEntity<Poliza> cancelar(@PathVariable Long id) {
52          Poliza poliza = service.cancelar(id);
53          return ResponseEntity.ok(poliza);
54      }
55  }

```

```
1  package com.web;
2
3  import com.domain.Riesgo;
4  import com.service.PolizaService;
5  import lombok.RequiredArgsConstructor;
6  import org.springframework.http.ResponseEntity;
7  import org.springframework.web.bind.annotation.*;
8
9  @RestController
10 @RequestMapping("/riesgos")
11 @RequiredArgsConstructor
12 public class RiesgoController {
13
14     private final PolizaService service;
15
16     @PostMapping("/{id}/cancelar")
17     public ResponseEntity<Riesgo> cancelar(@PathVariable Long id) {
18         Riesgo riesgo = service.cancelarRiesgo(id);
19         return ResponseEntity.ok(riesgo);
20     }
21 }
```

2. Configuración y Acceso a la Aplicación